SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

FACULTY OF COMPUTING.

WEB SECURITY- IE2062

IT22121592 – HERATH H.M.C.H.K

Y2S2

Subdomain ‘content.product.bigcommerce.com’

Subdomain Enumeration:

```
+ Target IP:          34.111.138.51
+ Target Hostname:    content.product.bigcommerce.com
+ Target Port:        443
---------------------------------------------------------------------------
```

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ dig content.product.bigcommerce.com @8.8.8.8

; <<>> DiG 9.19.21-1-Debian <<>> content.product.bigcommerce.com @8.8.8.8
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 29993
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;content.product.bigcommerce.com. IN    A

;; ANSWER SECTION:
content.product.bigcommerce.com. 525 IN CNAME   5716810793353216-content.customer.pendo.io.
5716810793353216-content.customer.pendo.io. 255 IN CNAME cdn-cname.pendo.io.
cdn-cname.pendo.io.     255     IN      CNAME   51.138.111.34.bc.googleusercontent.com.
51.138.111.34.bc.googleusercontent.com. 3600 IN A 34.111.138.51

;; Query time: 224 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Fri Oct 18 15:10:58 +0530 2024
;; MSG SIZE  rcvd: 205
```

Vulnerability title.

Insecure Direct Object Reference (IDOR)

Vulnerability description

Insecure Direct Object Reference (IDOR) a type of security vulnerability, in access control systems allows attackers to manipulate input parameters to gain access to resources or objects that are meant to be restricted from them. This vulnerability arises when a web application reveals implementation details like files, database records, and URLs, without performing authorization validations.

Understanding the Mechanisms of Insecure Direct Object References (IDOR)

Direct Access to objects.

- In web applications resources can be reached through URLs containing codes. For instance, a link could appear as follows: https://example.com/profile?id = 123.
- The unique identifier (123, in this instance) matches up directly with a resource, such, as a user profile.

Lack of Access control checks:

- If the program doesn't verify if the user is allowed to access the resource they're requesting correctly an intruder could change the identifier in the web address.
- Changing the identifier from id number 123, to id number 124 could potentially enable an actor to gain access to the profile of a different user under the assumption that the id number 124 corresponds, to a legitimate user account.

Exploitation:

- Attackers could potentially take advantage of Insecure Direct Object References (IDORs) by guessing or cycling through identifiers such, as user IDs or document IDs to gain access, to data or perform actions.

- They might access data and change information that only authorized users should be able to do.

A Sample Situation

Users can access their order information, on the website by visiting https://example.com/orders?id= 456

If a user has permission to access their orders and the application fails to verify if they are the rightful owner of the order. This could allow an attacker to tamper with the ID parameter.

By modifying the website link to https://example.com/orders?id= 457 an unauthorized person might be able to view the order information of a user.

Affected components.

In web applications weaknesses, like Insecure Direct Object Reference (IDORs) could potentially impact parts of the system.

The backend of a web application:

- Database records: Any entity in a database that can be easily accessed with a known identifier (such as user IDs or order numbers) is susceptible, to IDOR attacks. For accessing a user's profile or purchase history without authorization verifications.
- File storage: When files are stored in a way that can be easily accessed through paths, on the server it leaves room for exploitation. For example, if users can retrieve files using IDs in the URL there is a risk that they may gain access, to files owned by users.

RESTful application programming interfaces (APIs)

- API Endpoints that provide access, to resources using identifiers (for example GET/api/users/123) may have security vulnerabilities if they do not implement

authorization checks potentially allowing an attacker to manipulate the identifier and gain access, to other users' information.

- Performing operations like creating data entries or reading and updating existing content through APIs can pose security risks if the app fails to verify user permissions, for the object being accessed.

Web application frameworks

- Several web frameworks such, as Django and Ruby, on Rails offer ways to create URLs that directly link to database entries. If proper access controls are not established by developers when using these frameworks it can lead to security risks.

Fronted components

- When using JavaScript, in client-side scripts to create URLs or interact with APIs based on user input, without validation checks in place can lead to IDOR vulnerabilities being exposed. For instance, suppose a JavaScript application permits users to access records; in that case, a direct reference might be revealed if permission checks are not conducted.
- HTML forms that have fields, for identifiers, such as user IDs, without validation may result in Insecure Direct Object References (IDOR) if users can manipulate these fields.

Authentication and session management

- Session Cookies: When session identifiers or tokens are not adequately protected from access or manipulation, by actors it can lead to potential security risks such, as IDOR (Insecure Direct Object Reference) attacks.
- Authentication Logic: The components that handle user authentication and authorization could be a vulnerability if they do not effectively enforce access controls.

File upload components

- File Upload User Interfaces: If the software permits users to upload files using naming conventions or accessible identifiers and if users can then access files they are not supposed to access; this can result in Insecure Direct Object Reference (IDOR) vulnerabilities.

External services and third-party integration

- Applications that rely on third-party services could potentially be, at risk of IDOR vulnerabilities if they directly access resources on these services without verifying user permissions.

Content management systems (CMS)

- Content Management Systems (CMS) often include functionalities that enable users to generate content as well as modify or remove existing content items, within the system interface itself rather than using direct references such, as post IDs that could potentially expose them to IDOR attacks.

Impact assessment.

Evaluation of the When dealing with Insecure Direct Object Reference (IDOR) vulnerabilities can have an impact on both companies and the people using their services or products. Let's delve into the consequences of these vulnerabilities being taken advantage of.

Data Exposure:

Picture a situation where a malicious individual manages to obtain details such, as social security numbers or financial information by manipulating URLs or parameters to access data designated for different user use cases. When confidential information is exposed inappropriately like this it can have an impact on the trust relationship, between customers and the organization.

Data integrity:

Imagine the implications when a hacker not only reads but modifies data – for instance, altering someone's address or order information, without consent. This unauthorized tampering could result in chaos and mistakes within business procedures. Cause inconvenience, for the individuals affected.

Legal and regulatory consequences:

Businesses are required to follow regulations to safeguard individual's personal data from harm and misuse. If an IDOR vulnerability causes a data breach to occur the consequences could involve penalties and legal disputes. Imagine a situation where an upset client decides to take measures due, to their information being disclosed; the resulting financial and reputational repercussions could be significant.

Operational impact:

An IDOR vulnerability could cause disruptions, to services like when important data is exposed leading the company to close some parts of its app, for repairs. This does not strain resources. Can also annoy users who rely on the service.

Financial impact:

Dealing with a breach can come with expenses such, as hiring assistance and public relations services to handle the aftermath effectively. Moreover losing customer trust may result in a drop in sales, it's akin to throwing money.

User impact:

It can be quite unnerving, for individuals using the service when they realize that their data could potentially be viewed or altered by parties. This lack of control, over their information may lead them to hesitate in utilizing the service in the future.

Long-term impact:

Following a breach incident occurs within a company or organization; it may come under examination by bodies and customers alike who seek assurances, on future data protection measures in place to prevent recurrences of such incidents, in the future. This heightened level of scrutiny could result in a business environment and the process of restoring a damaged reputation may extend over several years.

Here are some possible effects to consider:

- Data Breach Concerns Arising from Unauthorized Personal Data Access.
- Maintaining Data Integrity is crucial to avoid the risk of missing data.
- Legal Ramifications; Penalties and legal action resulting from noncompliance.
- Operational Disruption occurs when there are service interruptions while attempting to resolve the problem.
- Financial Loss can result from expenses related to fixing issues and a decrease, in earnings.
- Concerns, about user privacy, often stem from feelings of vulnerability and a loss of trust, in the system.
- Facing scrutiny, over a period leads to supervision and difficulties, in rebuilding one's reputation.

Steps to reproduce:

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ dig content.product.bigcommerce.com @8.8.8.8

; <<>> DiG 9.19.21-1-Debian <<>> content.product.bigcommerce.com @8.8.8.8
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 29993
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;content.product.bigcommerce.com. IN    A

;; ANSWER SECTION:
content.product.bigcommerce.com. 525 IN CNAME   5716810793353216-content.customer.pendo.io.
5716810793353216-content.customer.pendo.io. 255 IN CNAME cdn-cname.pendo.io.
cdn-cname.pendo.io.     255     IN      CNAME   51.138.111.34.bc.googleusercontent.com.
51.138.111.34.bc.googleusercontent.com. 3600 IN A 34.111.138.51

;; Query time: 224 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Fri Oct 18 15:10:58 +0530 2024
;; MSG SIZE  rcvd: 205
```

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ nslookup content.product.bigcommerce.com 8.8.8.8
Server:         8.8.8.8
Address:        8.8.8.8#53

Non-authoritative answer:
content.product.bigcommerce.com canonical name = 5716810793353216-content.customer.pendo.io.
5716810793353216-content.customer.pendo.io      canonical name = cdn-cname.pendo.io.
cdn-cname.pendo.io      canonical name = 51.138.111.34.bc.googleusercontent.com.
Name:   51.138.111.34.bc.googleusercontent.com
Address: 34.111.138.51
```

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ curl -I https://content.product.bigcommerce.com
HTTP/2 403
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: private, max-age=0
content-length: 300
content-type: application/xml; charset=UTF-8
date: Fri, 18 Oct 2024 09:43:34 GMT
expires: Fri, 18 Oct 2024 09:43:34 GMT
server: istio-envoy
x-guploader-uploadid: AHmUCY3PvBby0xuzDLPM3f4qYV3mKVhyLtusUIDvkiEAAgjolGjMrwh4NJbM5Mqz4ft9tWMBTci-4ipKZg
x-envoy-upstream-service-time: 32
via: 1.1 google
strict-transport-security: max-age=63072000
alt-svc: clear
```

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ sqlmap -u "https://content.product.bigcommerce.com" --dbs
        ___
       __H__
 ___ ___[)]_____ ___ ___  {1.8.9#stable}
|_ -| . [)]     | .'| . |
|___|_  [']_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to
onsible for any misuse or damage caused by this program

[*] starting @ 15:14:03 /2024-10-18/

[15:14:04] [INFO] testing connection to the target URL
[15:14:05] [WARNING] potential permission problems detected ('AccessDenied')
[15:14:05] [WARNING] the web server responded with an HTTP error code (403) which could interfere with the results of the tests
[15:14:05] [INFO] checking if the target is protected by some kind of WAF/IPS
[15:14:06] [INFO] testing if the target URL content is stable
[15:14:06] [INFO] target URL content is stable
[15:14:06] [CRITICAL] no parameter(s) found for testing in the provided data (e.g. GET parameter 'id' in 'www.site.com/index.php?id=1')
[15:14:06] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 3 times

[*] ending @ 15:14:06 /2024-10-18/
```

Proposed mitigation or fix:

When addressing Insecure Direct Object Reference (IDOR) it's crucial to follow security practices to limit users' access, to authorized resources effectively. Here are some suggested strategies and solutions to tackle IDOR vulnerabilities with care;

Implement strong access controls:

- Always verify the user's authorization before granting access, to any content or service to guarantee that they possess the necessary permissions or privileges to view or interact with the item in question.
- Implement Role Based Access Control (RBAC) to handle user permissions according to their designated roles reducing the chances of individuals gaining access, to information.

Use indirect object reference:

- Of revealing identifiers (such, as user IDs) opt, for tokens or hashed values instead. For instance, utilize a randomized string or hash to symbolize the asset rather than a consecutive ID (for example substitute id=123 with token = abc123xyz).
- Maintaining Tables: Keep a table that converts to understand labels, into references and make sure to conduct access validations according to this table.

Input validation and sanitization:

- Make sure to check the input by setting up validation measures to confirm that the data provided by users is accurate and matches the expected formats.
- Make sure to clean up the data by sanitizing any information that could be used in identifiers to avoid tampering.

Set up logging and monitoring functionality:

- Keep track of audit logs to record all access attempts made, whether successful or unsuccessful as this can assist in identifying any behavior or potential exploitation of IDOR vulnerabilities.
- Utilize security monitoring software to observe access patterns and highlight any behaviors that could signal an IDOR attack.

Ensure to carry out security testing.

- Frequent Security Testing Practice; Conduct security tests to detect vulnerabilities, in the system and address ID-related flaws early on to prevent exploitation, by malicious actors.
- Make sure to perform reviews of the code with a focus placed on access control mechanisms to guarantee their accurate implementation.

User education and awareness:

- Security Training Instructions: Conduct training sessions to educate developers about the significance of coding methods, with a focus, on access control. The potential dangers linked to IDOR vulnerabilities.
- Make sure to inform users about how crucial it is to protect your accounts and spot phishing scams that may result in entry.

Secure session management:

- Secure session tokens should be made inactive either upon logout or, after a period of inactivity.
- Ensure that session tokens are limited to actions or resources to reduce the impact in case a token is compromised.

Regular Software Updates:

- Remember to update all parts of your application setup such, as libraries and servers to address any known security risks.

Ensure the utilization of Content Security Policies (CSP):

- Utilize Content Security Policy headers to stop content from loading and safeguard, against attack methods like cross-site scripting (CSS) often exploited in conjunction, with IDOR weaknesses.

In a nutshell here's the summary:

By following these measures companies can greatly lower the chances of IDOR vulnerabilities and safeguard sensitive data, from entry. Key elements of a security plan include enforcing access controls employing indirect references conducting comprehensive tests and fostering a culture of security awareness.